Министерство науки и высшего образования Российской Федерации
Бюджетное образовательное учреждение высшего образования
«Казанский национальный исследовательский технический университет
им. А.Н. Туполева КАИ»

Кафедра автоматизированных систем обработки
информации и управления

З.М. ГИЗАТУЛЛИН

Методические указания к лабораторной работе №5
по дисциплине «Интеллектуальные системы поддержки принятия решений»
на тему «Построение и анализ рекомендательной системы»

Казань – 2024

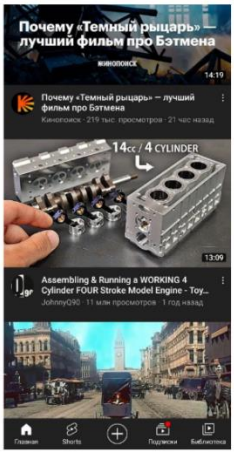
ЛАБОРАТОРНАЯ РАБОТА №5

Построение и анализ рекомендательной системы

Цель: привить практические навыки построения и работы с рекомендательными системами.

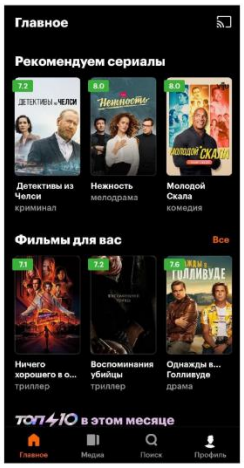
Сведения из теории

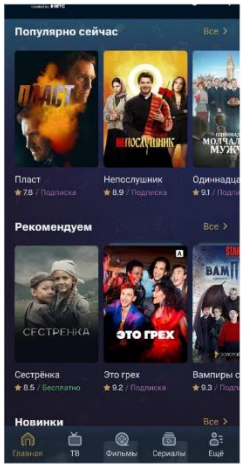
Рекомендательные системы окружают нас почти во всех сервисах, которые мы используем каждый день. Давайте посмотрим, а что же можно рекомендовать? Пожалуй, самой известной видео-платформой, которая имеет свою рекомендательную систему, является YouTube [1].



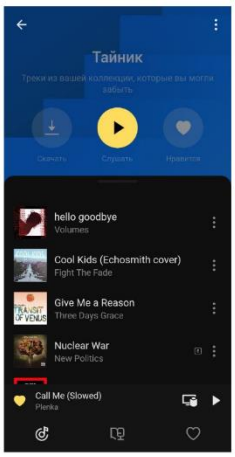


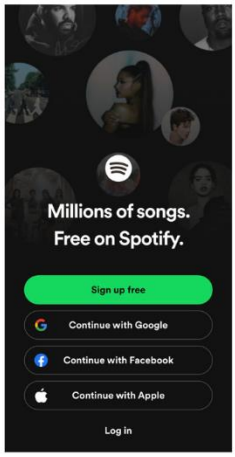
Раз мы заговорили о видео-хостингах, давайте сразу вспомним и об онлайн-кинотеатрах. На сайте любого онлайн-кинотеатра Вы можете увидеть несколько разных подборок с фильмами и сериалами, а одна из них будет персонализирована именно под Вас.





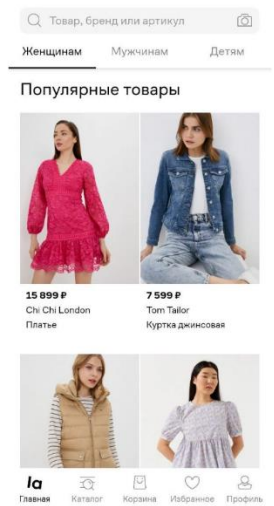
Вероятно, в свободное время или во время рабочих дел Вы любите послушать музыку. В музыкальных стриминговых сервисах также есть место для рекомендаций. Рекомендации в таких сервисах обычно представляют собой заранее сгенерированные плейлисты, либо могут иметь формат радио – когда очередная музыкальная композиция выбирается "на лету".





Рекомендации играют огромное значение в онлайн-магазинах. С помощью них не только Вы можете найти подходящий для себя товар, но и магазин заработать на большем числе продаж.





Как Вы уже поняли, рекомендовать можно многое: текстовые статьи; книги; посты, группы и друзей в социальных сетях; объявления о покупке/продаже товаров; кафе и рестораны; события и мероприятия; вакансии и др.

Что общего во всех этих примерах, когда сервису нужны хорошие рекомендации?

У сервиса должно быть много разнообразного контента (товаров), чтобы было ЧТО рекомендовать. У сервиса должна быть большая аудитория, чтобы было КОМУ рекомендовать.

Основная задача рекомендательных систем – упростить поиск релевантных объектов и обогатить пользовательский опыт.

Пример

Для иллюстрации работы алгоритмов рекомендательных систем (коллаборативная фильтрация), используется урезанный датасет оценок пользователей для фильмов MovieLens. Этот датасет является классическим бенчмарком для оценки качества рекомендательных систем в научной среде. Загрузим этот датасет при помощи ряда стандартных команд по ссылке [1]: https://colab.research.google.com/drive/1aignzmtgTjC8HErXwUJzWlqDcmR8NdCg?usp=sharing#scrollTo=2zBGscoclHtA

Далее загружаются библиотеки для рекомендательных систем. Загружаются датасет и сортируется по пользователям и времени. Рассматриваются основные характеристики загруженного датасета и можно посмотреть информацию о фильмах в датасете. Например, отбираются только позитивные оценки и посчитаем отметки для каждого фильма: n_x и n_y.

```
user_likes = ratings_pos.groupby('userId')['itemId'].agg(list)
user_likes
```

```
userId
1       [3186, 1270, 1721, 1022, 1836, 3408, 2804, 120...
2       [1198, 1210, 1293, 2943, 1225, 1193, 318, 3030...
3       [2858, 1968, 1961, 1266, 1378, 1379, 3671, 590...
4       [1097, 3468, 480, 260, 1198, 1387, 2028, 2366,...
5       [908, 919, 1250, 2858, 2997, 2770, 2355, 2908,...
                              ...
6036    [1721, 1883, 2858, 2997, 2710, 223, 3203, 1093...
6037    [1267, 2028, 3148, 858, 562, 912, 1221, 923, 2...
6038    [1210, 2146, 356, 1079, 1148, 3548, 3088, 232,...
6039    [282, 111, 2067, 930, 3022, 3088, 2804, 955, 1...
6040    [858, 593, 2384, 1961, 2019, 573, 213, 3111, 3...
Name: itemId, Length: 6038, dtype: object
```

Посчитываются величины n_{xy}, а также убираются те фильмы, которые встречались друг с другом менее 100 раз. Так как датасет достаточно объемный, эта мера поможет не рассматривать огромное количества неинформативных пар. Теперь объединяются все счетчики вместе и строятся рекомендации для itemId_x, отсортированные по убыванию величины score.

```
[ ] counts_merged = pd.merge(counts_pair, counts, left_on='itemId_y', right_index=True)
    counts_merged['score'] = counts_merged['pair_cnt'] / counts_merged['cnt']
    counts_merged.sort_values(by=['itemId_x', 'score'], ascending=[True, False], inplace=True)
    counts_merged.head()
```

	itemId_x	itemId_y	pair_cnt	cnt	score
1942	1	783	129	162	0.796296
1944	1	1566	163	205	0.795122
1940	1	2294	252	330	0.763636
62596	1	1023	125	166	0.753012
1946	1	1907	242	323	0.749226

Рассмотрим топ-30 рекомендаций для каждого фильма.

```
Рекомендации для Toy Story (1995):
==================================================
01      Hunchback of Notre Dame, The (1996)
02      Hercules (1997)
03      Antz (1998)
04      Winnie the Pooh and the Blustery Day (1968)
05      Mulan (1998)
06      Sword in the Stone, The (1963)
07      Robin Hood (1973)
08      Muppet Christmas Carol, The (1992)
09      Tarzan (1999)
10      Great Muppet Caper, The (1981)
11      Muppets Take Manhattan, The (1984)
12      Pinocchio (1940)
13      Home Alone (1990)
14      American Tail, An (1986)
15      Dumbo (1941)
16      Cool Runnings (1993)
17      Charlotte's Web (1973)
18      In & Out (1997)
19      Anastasia (1997)
20      That Thing You Do! (1996)
21      Muppet Movie, The (1979)
22      Sleeping Beauty (1959)
23      Beavis and Butt-head Do America (1996)
24      Cinderella (1950)
25      Blast from the Past (1999)
26      Aladdin (1992)
27      Beauty and the Beast (1991)
28      Lion King, The (1994)
29      Jungle Book, The (1967)
30      Toy Story 2 (1999)
```

В общей схеме работы рекомендательных систем можно выделить следующие этапы (рис. 5.1):

- candidates – этап отбора кандидатов из всей коллекции сущностей items;

- ranking – этап ранжирования отобранных кандидатов;

- business rules – этап применения бизнес-правил.

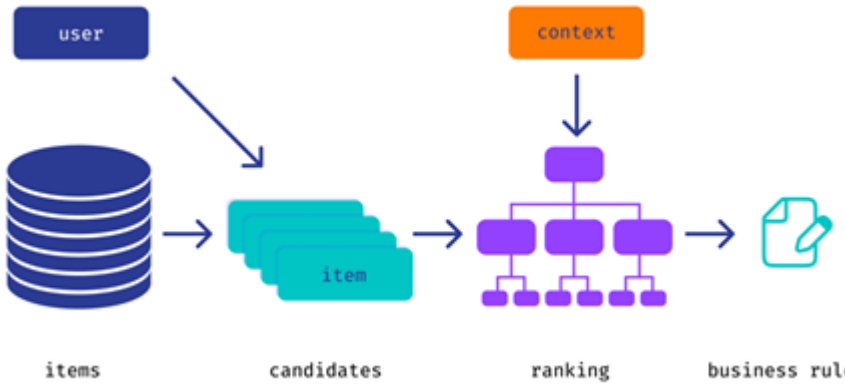


Рис. 5.1. Общая схема работы рекомендательных систем

Постановка задачи.

Пусть есть:

U – множество пользователей – users;

I – множество сущностей (товаров, документов и т.п.) – items.

R={(u,v):ruv} – множество взаимодействий/рейтингов/оценок.

Множество взаимодействий можно представить в виде одной разреженной матрицы R, где: строки ~ пользователи; столбцы ~ сущности; пересечение – рейтинги. На пересечении пользователя и товара записано какое-то их взаимодействие (feedback) (рис. 5.2).

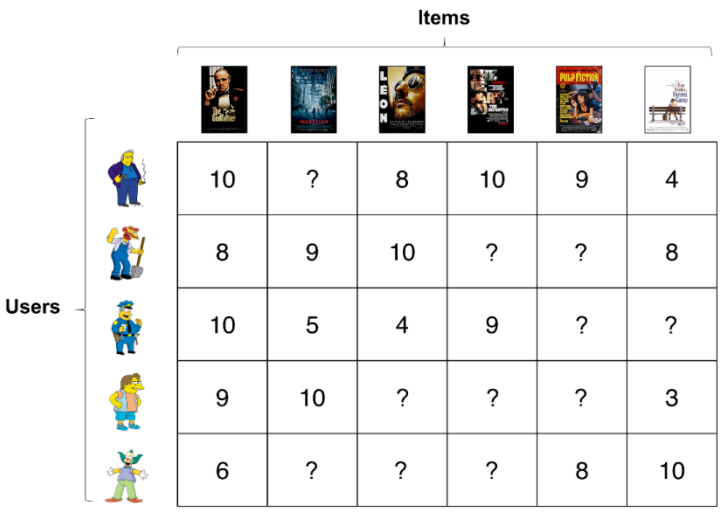


Рис. 5.2. Матрицей взаимодействий

Тогда мы хотим уметь решать следующие задачи:

предсказывать пропущенные элементы в матрице;

уметь оценивать похожесть пользователей/сущностей друг на друга;

уметь строить рекомендации для пользователей/сущностей.

Возникает естественный вопрос: как правильно выбирать значения в матрице R?

Существует два типа сигнала (feedback):

Явный – explicit: лайк/дизлайк; оценка; покупка; отзыв.

Неявный – implicit: клик/показ; глубина просмотра; добавление в корзину; поведение (например, поисковые запросы).

В качестве значений R можно использовать весовую сумму обоих типов сигналов.

Выделяют 4 основных типа рекомендательных систем:

Основанные на поведении пользователей (collaborative filtering)

Основанные на экспертных знаниях (knowledge-based)

Основанные на контенте (content-based)

Гибридные (hybrid)

В данной работе мы рассмотрим коллаборативную фильтрацию – группу алгоритмов, которые учитывают поведение пользователя в сервисе и его взаимодействие с рекомендательной системой. Такие алгоритмы считаются наиболее сильными и дающими наилучшее качество.

Алгоритмы коллаборативной фильтрации можно условно поделить на две группы:

Memory-based – алгоритмы, которые хранят всю матрицу взаимодействий R. Похожесть пользователей задается как похожесть строк матрицы, похожесть сущностей – похожесть столбцов матрицы.

Latent factor model-based – алгоритмы, которые выучивают векторные представления для пользователей и сущностей. С помощью этих представлений можно частично восстановить значения в матрице R. Похожесть между пользователями/сущностями задается как похожесть между соответствующими векторными представлениями.

item2item рекомендации (основан на похожести между товарами)

Попробуем решить следующую задачу. Мы хотим научиться рекомендовать для каждого фильма список похожих на него фильмов (на основании поведения пользователей). Возьмем все фильмы, которые понравились пользователям. Будем полагать, что фильмы, которые смотрят вместе одни люди, скорее будут похожими.

В качестве рекомендации пользователю после позитивной оценки некоторого фильма мы выведем целый список похожих фильмов, отранжированных по их похожести.

То есть, например, Вы посмотрели фильм "Звёздные войны: Эпизод V", и он Вам очень понравился, Вы поставили ему высокую оценку. Item2Item подход предлагает сразу же посмотреть, какие фильмы, как мы знаем, похожи на фильм "Звёздные войны: Эпизод V" с точки зрения предпочтений уже известных нам пользователей. В простейшем случае мы бы могли посмотреть, какие фильмы часто смотрят вместе со "Звёздными войнами", и просто предложить их, но мы воспользуемся более интересной идеей.

Для этого хорошо бы научиться измерять похожесть фильмов, соответственно рекомендациям, этими фильмами полученным. Для этого воспользуемся метрикой, которая называется PMI.

PMI (pointwise mutual information)

Пусть некоторый объект описывается множеством бинарных (в общем случае - категориальных) признаков. Для оценки связи между категориальными признаками часто используется мера PMI. Пусть есть 2 бинарных (или опять же - категориальных) признака: x и y.

Введём обозначения:

p(x,y) – вероятность встретить признаки x и y у одного объекта;

p(x) – вероятность встретить признак x у объекта;

p(y) – вероятность встретить признак y у объекта.

Рассмотрим случай, при котором x и y встречаются у объекта независимо друг от друга. Тогда:

$p(x,y)=p(x)\cdot p(y)$

То есть

$p(x,y)/p(x)\cdot p(y)=1$

Если же x и y не независимы, то

$p(x,y)=p(x)\cdot p(y|x)=p(y)\cdot p(x|y)$

То есть величина

$p(y|x)/p(y)=p(x|y)/p(x)=p(x,y)/p(x)p(y)$

демонстрирует нам степень связи между компонентами x и y. Например, если эта величина больше 1, мы можем смело утверждать, что признаки x и y часто встречаются вместе, поскольку вероятность встретить признак x при условии наличия признака y превышает вероятность встретить y произвольного объекта признак y вообще. И наоборот, если эта величина меньше 1, то вероятность встретить признак x у объекта с признаком y ниже, чем в среднем.

Не очень удобно работать с величиной, нейтральное значение которой равно 1, а качественный результат получается при сравнении именно с 1. Намного удобнее было бы перенести нейтральное значение в 0, а также выровнять шкалу. Это можно сделать при помощи логарифмирования полученного выражения. Таким образом, мы приходим к метрике PMI:

$PMI(x,y)=\log_2 p(y|x)/p(y)=\log_2 p(x,y)/p(x)p(y)=\log_2 p(x|y)/p(x)$,

Воспользуемся частым приемом математической статистики: в дискретном случае можно заменить вероятности на частотные оценки:

$p(x,y)=n_{xy}/N$, $p(x)=n_x/N$, $p(y)=n_y/N$,

тогда $PMI(x,y)=\log_2(n_{x,y}N/n_x n_y)$,

Где:

n_x – число объектов, которые имели признак x;

n_y – число объектов, которые имели признак y;

$n_{x,y}$ – число объектов, которые имели оба признака x и y;

N – общее число объектов.

На практике ради простоты вычислений можно сделать шаг назад: величина PMI(x,y) используется для сравнения (ранжирования) признаков x и

y, поэтому для простоты можно отказаться от логарифма и константы N и перейти к величине score(x,y)=$n_{x,y}/n_x n_y$

В нашем случае в качестве объекта будет выступать пользователь, а категорийным признаком – факт, что пользователь положительно оценил фильм.

Для лучшего понимания предложенной идеи оценки близости пользователей и объектов приводится такой **пример**: пусть есть три пользователя некоторого онлайн-кинотеатра: Паша, Света и Миша.

Каждый из них лайкает ровно половину просмотренных фильмов (Так уж случилось, они любят кино). Пусть, например, таблица с их лайками всех эпизодов "Звездных Войн" выглядит так (https://colab.research.google.com/drive/1aignzmtgTjC8HErXwUJzWlqDcmR8NdCg?usp=sharing#scrollTo=TcwBXoBgHUDG) (рис. 5.4.):

	Паша	Света	Миша
Эпизод I	0	1	0
Эпизод II	0	1	0
Эпизод III	1	0	1
Эпизод IV	1	0	1
Эпизод V	0	1	0
Эпизод VI	1	0	1



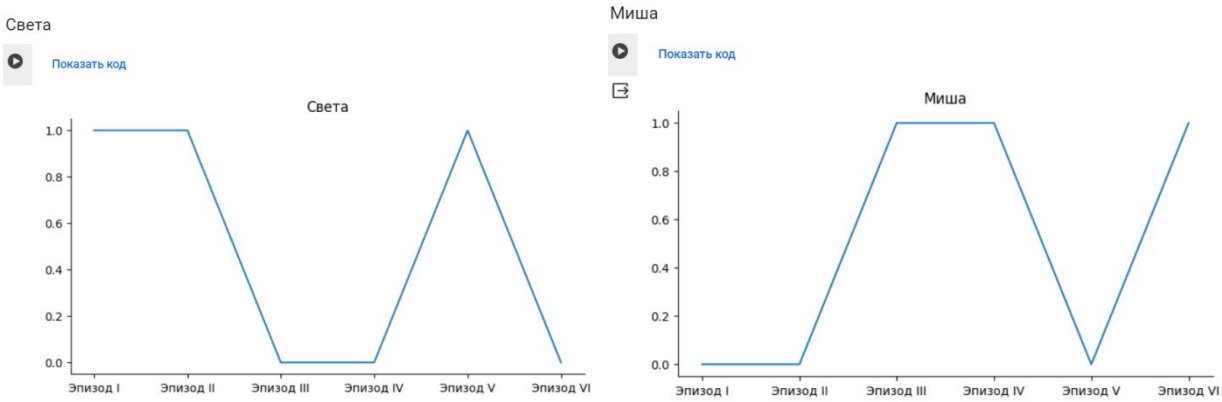


Рис. 5.4. Результат работы программы

Как мы видим, Паша и Миша всегда лайкали одни и те же эпизоды "Звёздных Войн", а вот мнение Светы радикально отличалось от мнения друзей - она лайкала все остальные эпизоды.

Что мы можем сказать в этом случае о похожести вкусов пользователей? Конечно, нам бы хотелось, чтобы наша метрика обнаружила максимальное сходство Пашиных вкусов с Мишиными и отличие от вкусов Светы.

Посчитаем для начала вероятность того, что Паша лайкнет некоторый случайный фильм: =0,5

А теперь узнаем, какова вероятность того, что Паша лайкнет некоторый фильм, при условии того что его уже лайкнула Света: = 0.

Совершенно не удивительно, ведь они не лайкнули совместно ни одного фильма. А что будет, если мы посчитаем такую же величину для Миши? =1

Тоже вполне очевидный результат. Посчитаем теперь отношение вероятностей: #Паша и Света = 0; #Паша и Миша = 2.

Логарифмируя эту величину, мы и получим PMI, соответственно = -16.61; =1.

PMI измеряет совместимость людей намного лучше, чем гороскоп.

Задание:

1) Изучить теоретический материал лабораторной работы.

2) Необходимо реализовать рекомендательную систему (табл. 5.1 или инициативная тема). Провести анализ и интерпретировать полученные результаты.

3) Оформить отчет по выполненной работе.

Таблица 5.1.

Варианты заданий

Вариант	Задание
1	Рекомендательная система по подбору высших учебных заведений для абитуриентов [2]
2	Рекомендательная система по фильмам [3]
3	Рекомендательная система на основе анализа поведения пользователей на сайте [4]
4	Рекомендательная система для выбора книг [5]
5	Рекомендательная система в сфере логистики [6]
6	Рекомендательная система музыки
7	Рекомендательная система новостей
8	Рекомендательная система веб-сайтов
9	Рекомендательная система в интернет-магазине
10	Рекомендательная система выбора услуг
11	Рекомендательная система в социальных сетях
12	Рекомендательная система в спортивных соревнованиях
13	Рекомендательная система тематических материалов
14	Рекомендательная система в сфере авиационных систем
15	Рекомендательная система системы ресторанов [7]

Контрольные вопросы

1. Постановка задачи рекомендательных систем.
2. Опишите основные этапы работы рекомендательных систем.
3. Что такое матрица взаимодействий?
4. Назовите основные типы рекомендательных систем?
5. Основные идеи коллаборативной фильтрации?
6. В чем идея оценки близости пользователей и объектов?
7. Как обосновывать рекомендации?
8. Как бороться с проблемой холодного старта?
9. Как учитывать неявные предпочтения пользователю?
10. Как учитывать связи между пользователями?

Список литературы

1. Рекомендательные системы [Электронный ресурс] // Google Colaboratory — бесплатная среда для разработки и выполнения программного кода в облаке. URL: https://colab.research.google.com/drive/1aignzmtgTjC8HErXwUJzWlqDcmR8NdCg?usp=sharing#scrollTo=0cb20066-c112-44fb-9585-a18d74098478 (дата обращения: 28.02.2024). Режим доступа: свободный.

2. https://cyberleninka.ru/article/n/postroenie-rekomendatelnoy-sistemy-po-podboru-vysshih-uchebnyh-zavedeniy-dlya-abiturientov

3. https://www.kubsu.ru/sites/default/files/users/39916/portfolio/4it_fpm_ivanovea_2023.pdf

4. https://habr.com/ru/companies/otus/articles/764680/

5. https://habr.com/ru/companies/otus/articles/764222/

6. https://habr.com/ru/companies/epam_systems/articles/526748/

7. https://www.hse.ru/edu/vkr/474479543